

Christian Peterson, Shreesh Dassarkar, Jack Roberge
DS 4420
Dr. Gerber
December 4, 2025

Project Report: Advanced Analysis & Prediction of NFL Analytics

Github: <https://github.com/sdassarkar/DS4420-Final-Project>

Introduction:

Data analysis and prediction techniques have been disrupting sports management techniques powered by traditional statistical analysis. In the past, player and team-based statistics that management used (and the public viewed) were measured around absolute or relative frequencies; especially in baseball, this essentially would produce metrics that were unable for accurate comparisons (Cochran, 2010). Since the late 1990s/early 2000s, data driven methods have been growing and displacing traditional executive analysis due to their strong performances, which has benefited from enhancements in data collection, storage, and computational power (Cochran, 2010). This wider form of adoption has included many professional sports organizations creating internal teams that specialized in deeper statistical analysis and data driven decision making. The caveat is that this adoption occurred firstly in the MLB, where teams “widely accepted Sabermetrics as an effective way to evaluate players and make strategic decisions” (Pilgrim, 2023). Contrarily, adoption in the NFL for example was lagged until the Philadelphia Eagles’ 2018 Super Bowl victory, which was driven by these deeper analytical techniques (London Post, 2024). In addition, the magnitude of this transformation still has not reached levels observed in the MLB. To gain a more accurate understanding of how predictive analytics can be harnessed and improved by professional NFL organizations, we have chosen to apply two specific Machine Learning concepts - MLP (maybe CNN applied first) Neural Networks and Time Series Analysis - to perform an analysis of our own. In this project design, we therefore decided to pair two specifically complex methods with a voluminously descriptive dataset.

At a high level, the aim of our project is to predict NFL play calls based on relevant factors such as field position, opponents and positional strengths, prior play calls and success rates, etc. Specifically, we aim to use Neural Networks and Bayesian Linear Regression. There is some literature that exists employing some of these methods in the context of the NFL, but there are a few key points of differentiation that we hope to achieve.

With regard to the usage of Neural Networks, we found a few papers detailing the application in an NFL context. One applied a Neural Network to establish a new metric for the quarterback position - Expected Points Contributed by Passing Offense - in order to add more nuance to quarterback evaluations (Husowitz et al., 2024). While insightful, our goals revolve

around predicting specific play calls, not specific positional success. Concurrently, we found 2 articles that revolve around using Neural Networks for playcall prediction. The first article used a neural network to predict in real time if a play would be a pass or a run (Fernandes et. al, 2020). While quite accurate, with a score of 75.3%, a binary predictor is not too insightful - we therefore hope to add more dynamic classes that encapsulate field position and distance. More specifically, this would represent categorical depictions such as left run, right run, short left pass, deep right pass, and more. The data we found is quite robust and should enable us to meet these objectives. In the next section, we will discuss how this Neural Network-based classification will branch off into our Time Series models for further predictions.

Another portion of our project targets the usage of a Bayesian linear regression model to model Expected Points Added (EPA) given a certain prior belief of how this variable behaves, a model for which there is limited foundational support for in literature. Prior work does involve modeling EPA, but takes a slightly different approach. One source employed a multinomial logistic regression model to estimate EPA (Yurko et al., 2019). The model first took probabilities of seven possible scoring events conditional on the current game situation. Then, the “expected points” were determined by multiplying each event's predicted probability by its associated point value and summing them up. EPA is the difference between pre and post snap values. This is an intuitive way to model EPA, but we want to take a slightly different approach. To our knowledge, no prior work specifically employs a Bayesian approach to regression when it comes to modeling EPA. We believe this method could be advantageous as it allows us to quantify uncertainty better (by allocating a range of values for EPA as opposed to a static value) and incorporate domain knowledge through priors (intuitively we know EPA should center around 0). Moreover, we believe this approach provides a more complete probabilistic picture of how valuable a play could be.

Methodology & Data Collection:

Our dataset consists of NFL play-by-play data from the 2019 season, sourced from a DoltHub repository. Each observation represents a single offensive play, all the way from kickoff to the end of each game. The dataset includes a plethora of situational features such as field position (yard line, side of field), down and distance, specifications on goal to go situations (within 10 yards of the end zone), formation indicators (shotgun, no-huddle), game context (quarter, time remaining, score), and the outcome of the play (yards gained, play location). We used a variety of these features in our MLP and Bayesian Linear Regression models.

For the MLP, we aggregated a couple different outcome features mentioned above to define (and later predict one of) 6 play classifications, which are:

- | | |
|-------------------|--------------------|
| ● Left Run | ● Right Run |
| ● Short Left Pass | ● Short Right Pass |
| ● Deep Left Pass | ● Deep Right Pass |

Given the relatively small number of features used to predict the output (10 situational features) we initially leaned towards a simpler neural network with one hidden layer activated by the ReLU function, and our output passed through the softmax function. However, for the sake of completion, we also explored what a deeper network would look like. After implementing a network with two, three, and four hidden layers, we found that increasing the depth of the neural network seemed to have no meaningful impact on the accuracy of our model. Furthermore, given the fully interconnected nature of our MLP model and the limited computational power available on our home computers, increasing the depth drastically increased the time required for the gradient descent algorithm to converge (if it ever would). With minimal to no returns, increased execution times, and a huge increase in complexity, we felt it would be most effective to stick with a single hidden layer. Additionally, as we had chosen to perform hyperparameter tuning to optimize our model's performance, such tuning of the model over all combinations of widths, step sizes, and epochs across multiple hidden layers would have exponentially increased the time required to train the model. As Dr. Gerber has emphasized throughout the semester, if there is no benefit to making a model more complex, keep it simple!

In addition to the choice of a single hidden layer, we eventually settled on using the 'Leaky' ReLU activation function on the hidden layer as opposed to the standard ReLU function, allowing a small, non-zero value for negative inputs. The primary reason for this is since we have limited hidden nodes and only one hidden layer, we couldn't afford to lose information. The nature of our input data being a mix of encoded categorical features (0 or 1) and normalized continuous ones that could take negative values could lead to scenarios where ReLU would kill the learning. For instance, late game scenarios where the team's score is below average would lead to negative values. Depending on the categorical encodings, even positive weights may result in ReLU killing the signal. If this happens for a large proportion of late game scenarios, it harms the learning of the network. Leaky ReLU ensures stability even when specific feature combinations produce negative activations.

Given this was a multiclass classification problem at its core (using feature variables to predict the specific type of play as the output), we used the softmax function to convert the raw output of the neural network into a probability distribution over the six defined play types (allowing us to predict which is most likely). We then employ the cross-entropy loss function as our loss function.

We also chose Bayesian Linear Regression as our second approach to gauge an important variable in NFL analytics: Expected Points Added (EPA). EPA represents the expected points gained from a specific play that the offense runs. Intuitively, EPA could range from -2 (a safety on the play) to 6 (scoring a touchdown). Evaluating EPA complements our MLP's goal of predicting the play type by going further to analyze how a specific play, and a combination of multiple ones, can drastically change outcomes over time. At the same time, in order to model

how this variable changes per a specific play, we sought to implement Bayesian Linear Regression.

This enhanced version of Linear Regression is more advantageous than standard linear regression for two main reasons: the ability to assess uncertainty and the inclusion of prior beliefs. Bayesian Linear Regression could allow us to produce predictions of the EPA in distribution form, which could provide insights into the overall spread of this prediction. Contextually, this aspect is important since it is difficult to directly predict expected points in football due to the number of factors affecting a play and the ability to score. However, we can still incorporate certain features explicitly from the data to get some form of prediction. By including our prior beliefs about certain relevant features (down, yard line, etc.) that could influence EPA, we can assess how the model pulls the data away from this, which could either reinforce our predictions or highlight that modifications need to be made.

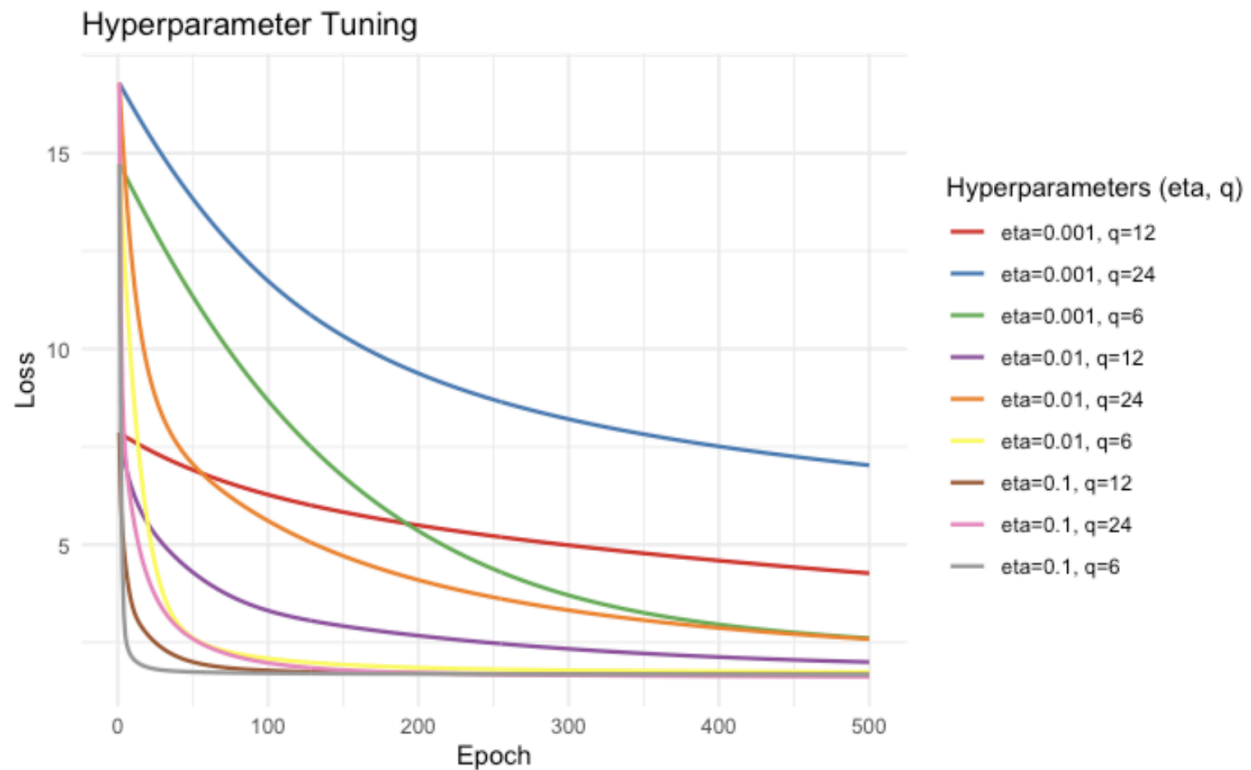
To take this all into account, we chose to implement a Normal Model with Normal priors for the weights and a Normal-Inverse Gamma prior for the unknown covariance matrix. We did this since EPA is a continuous feature and, when analyzing its general distribution, we found that it had a bell-curved shape with a mean hovering around 0 (and a spread mostly between -6 and +6). As for choosing the features that will influence EPA, we determined that the beginning yard line of the play, the offensive team's yards to go before the end zone is reached, amount of seconds remaining in the quarter, the down, and the specific type of play would all influence and assist in the prediction of this. For the unknown weights' Normal priors, we instantiated them from the following influences: the yard line would be a strong positive predictor in increasing the EPA, seconds remaining per quarter would do the same to a lesser degree, offensive yards to go would be a somewhat strong negative predictor, the categorical down would be a strong predictor in decreasing the EPA, the categorical play type (we removed certain types that were exclusively observed during a certain down, to prevent convergence issues) would have no average effect in increasing/decreasing the EPA, and the intercept would be close to 0 (EPA models this on average). Again, since we also did not know the covariance matrix, we used the Inverse Gamma distribution to estimate this. The combination between this estimate, and the combination between our Normal priors and collected play-by-play data, created a Normal likelihood. An optimized version of Gibbs Sampling was then utilized to estimate the conditional posteriors. For the prediction, a benefit of this approach is its equivalency to the basic Original Least Squares method, where we also can assess the overall distribution for each. The posterior mean of EPA is simply a weighted average between the OLS calculation and the prior belief about how the weights behave, all weighted by the covariance matrices, which creates usefulness.

Results & Interpretation:

We found the accuracy of our MLP model to be low at face value, but pretty decent when considering some more context. Given the six output classifications outlined earlier, a random guess would have yielded an accuracy of 1 in 6, or around 16.67%. With optimized hyperparameter tuning, our MLP model produced an accuracy of ~30.2% against test data. The fact that this is significantly higher than a random selection's accuracy indicates the model was able to perform some learning through the process and get an idea of which features strongly indicate certain plays. Given the complexity of the task here—using features to predict a process that involves an enormous amount of human intuition and situational randomness, it is rather unsurprising that there isn't overall high accuracy. This task of predicting play calls is made even further difficult by the level of granularity we assigned the model's predictions, as we asked the model to predict 1 of 6 play calls, contrasting previous literature that only predicted if a play was a pass or a run. We further see that the model's accuracy is notably better than simply predicting the most common play (short right passes), as this play is under a quarter of the data set, compared to our accuracy of over 30%.

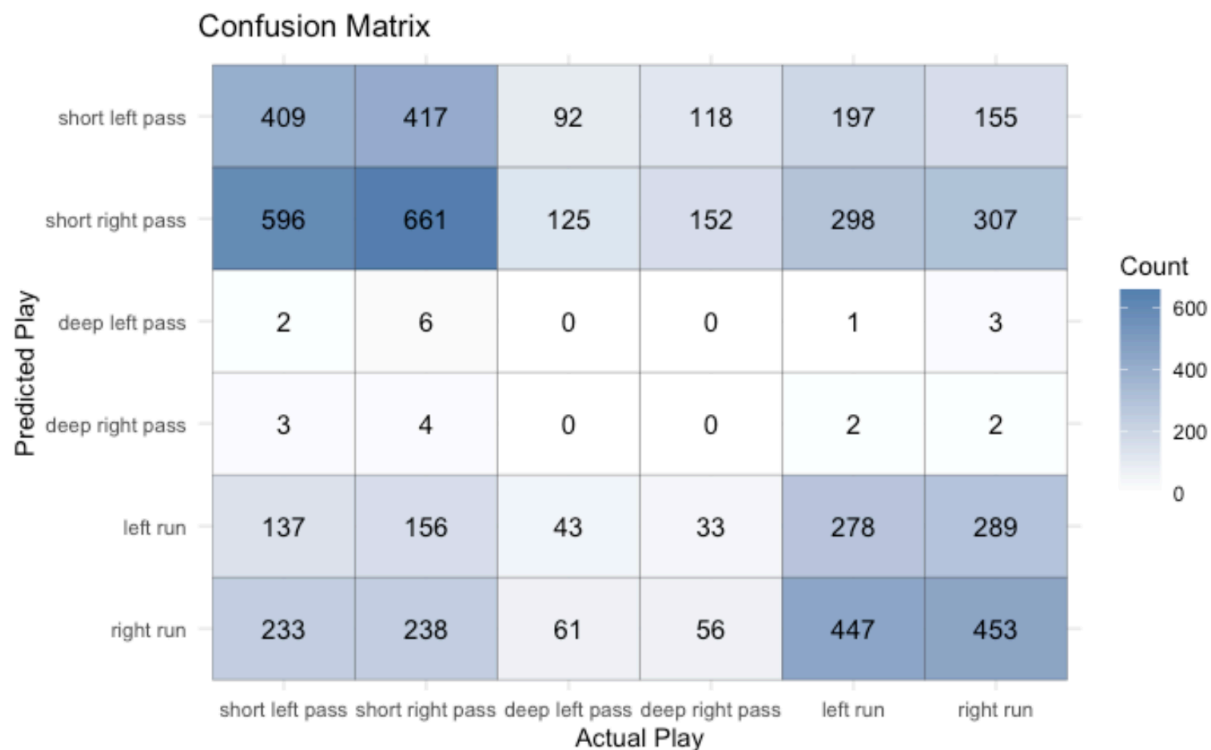
Due to the limited number of pre-play game state data that we had, it's likely that certain features used in the model became stronger predictors of the ensuing play call than they would be in reality, clouding the model's prediction of the ensuing play. For example, intuitively, whether an offense runs a huddle or no-huddle play is not as significant a determinant of the ensuing play as something like the specific formation or the number of each position on the field. However, since we don't have access to the latter features, more importance is placed on the huddle situation, whereas human game sense tells us otherwise.

From our hyperparameter tuning, we find that smaller step sizes (0.001, 0.01) are too small for the model's loss to effectively converge to optimal weight matrices, with the absolute value of the slope of the epoch vs loss curves associated with smaller step sizes remaining rather high even as we approach 500 epochs. Given the relatively sparse nature of our feature data after one-hot encoding the four categorical features and the large number of samples we are working with, it's not surprising that smaller step sizes are unable to find optimal weight matrices quickly enough in 500 epochs. We see a learning rate of 0.1 strikes a nice balance between overshooting the optimal weightings and getting stuck in local minima.



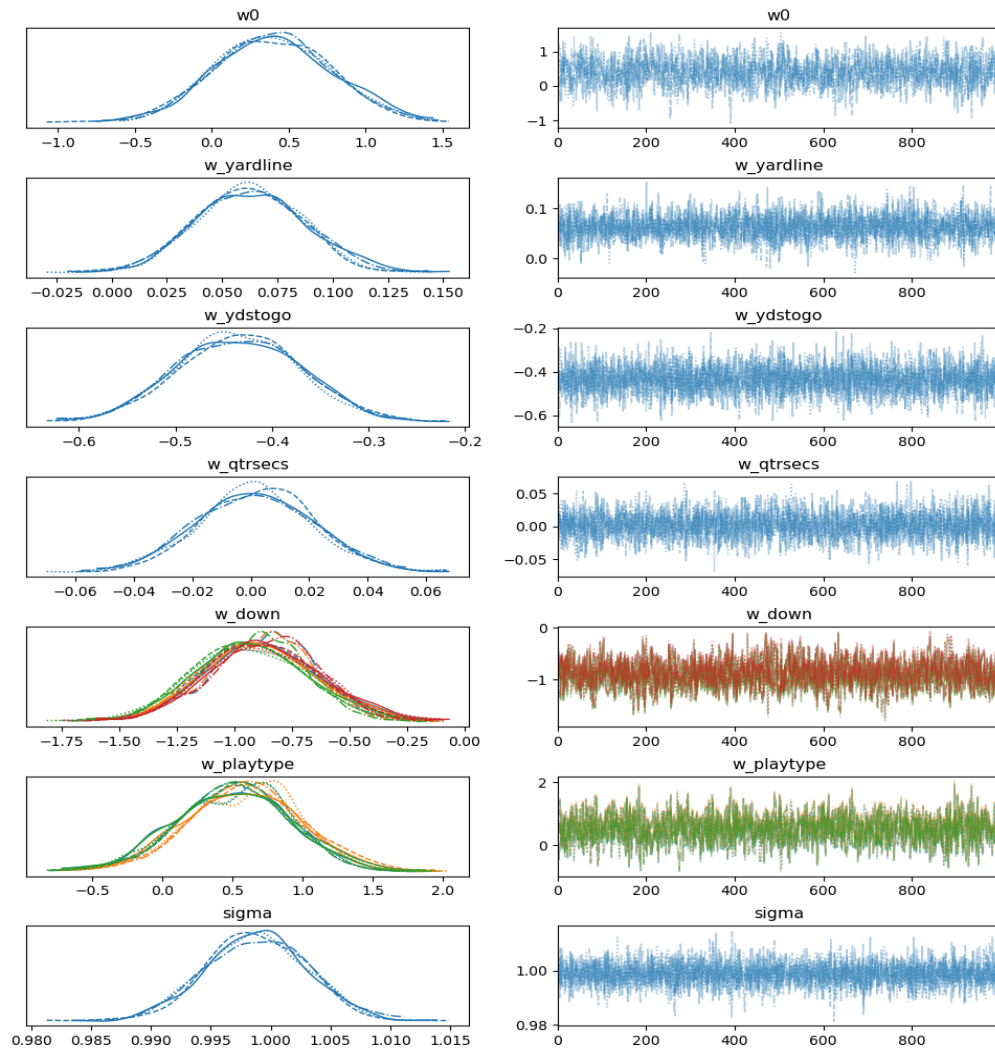
We also see from hyperparameter tuning that a wider hidden layer generally increases accuracy across all learning rates. Given that the number of inputs is 10 and there are 6 output classes, it's unsurprising that narrow hidden-layer models are unable to capture the nuance and complexity of football plays as effectively as a more complex hidden layer with 24 nodes. However, we also note that the number of nodes in a hidden layer should be somewhat related to the size of the inputs and the number of classes possible in the output. As such, a hidden layer significantly wider than this would likely fall prey to the classic overfitting trap, meaning that 24 is near the higher range of how many neurons are appropriate to include for the single hidden layer.

Because of our use of a non-linear activation function like leaky ReLU, the input features—play characteristics—are not linearly related to the output—type of play. This means that the magnitude or direction of the weight matrix coefficients has no practical implication for understanding how these features impact play classification. A different type of model (i.e. multivariate regression) would be needed in order to draw those types of conclusions (and this model would be unable to perform the type of multi-class classification we are performing here).



However, we can draw many meaningful conclusions from the confusion matrix output of our highest-performing model. From the multi-class confusion matrix, we first noticed that the model correctly predicted no instances of deep left passes or deep right passes, and predicted deep passes overall less than 20 times across both classes. Given how rare deep passes are overall (as evidenced by their infrequency in our data set), and how these passes are more so dependent on defensive coverages (data which we don't have access to), the model essentially dismissed these classes as noise and ignored them almost entirely. The most prominent error the model made was confusing a short right pass for a short left pass (and vice versa), which occurred a combined 1013 times in the training data. However, this is not indicative of poor model performance. Direction is usually based on defensive alignment, and since we don't have access to that in the data, the fact that the biggest mistake was confusing a type of short pass for another type of short pass is actually a compliment to the model's performance—it predicts the crux (short pass) correctly. Interestingly, when we disclude the left/right distinction from the model's classification (i.e., only test whether it can predict the type of play—short pass, deep pass or run—correctly), the model's accuracy improves to nearly 60%. Even more encouragingly, we'd argue that a mix-up between a short pass and a run (another frequent error) does still reflect some element of learning by the model. Because both of these plays have the same strategic intent—gaining a few quick yards—a mix-up here means the model has still been able to pick up on the strategic intent of the play from the given features, even if it didn't classify it perfectly.

Model Validity (Convergence):



A model can only be truly valid if the 4 Markov Chains we used in sampling, to estimate the posterior distribution, have converged. With the Effective Sample Size in the summary output, which gauges an amount estimate of independent samples that have statistical significance equal to our samples, the bulk and tail values are either equal to 1000 or greater; this means that autocorrelation is relatively low and the chains are producing independent-like samples. In terms of the R-hat values, which compare the variances between the chains to assess convergence, all are equal to 1.0; for convergence to be reached, these need to be around 1.0 or 1.01, which is satisfied in our model.

When analyzing the trace plots, we can first see the prior distributions for the weights that we defined (intercept, yard line, offensive yards remaining, seconds remaining in the quarter, down, play type, and sigma). We mostly see distributions following the same shape and spread, so we can assume that convergence has mostly been reached. However, there seems to be a small divergence in 2 of the chains for quarter seconds remaining, as well as larger divergences in the down and play type categorical variables. As for the trace plots, convergence is determined when

the 4 chains are varying in a way that makes the plot “fuzzy” after the number of iterations past the burn rate; based on these, we can see that convergence has been reached, with only minor divergences in the down feature. Therefore, based on the analysis from the correlation summary output and plots above, we can assume that the model’s sampling Markov Chains have reached relative convergence.

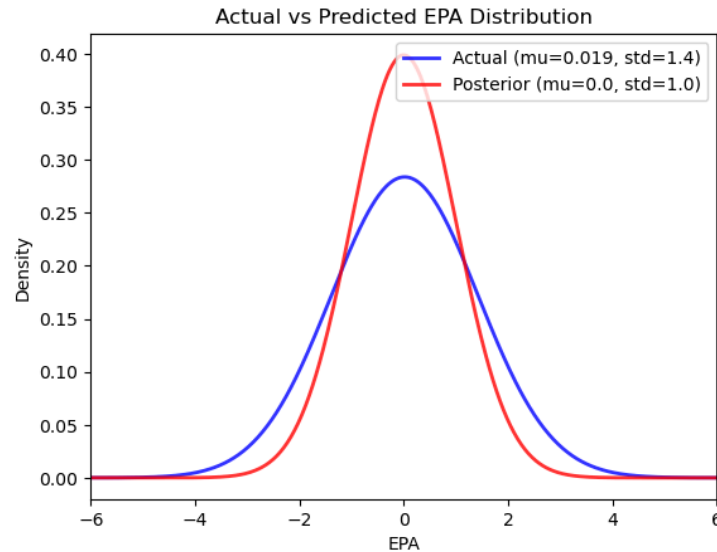
Feature Importance:

By analyzing how each of the chosen features influence the posterior distribution, we can develop a deeper understanding of how the weights of these features, via the prior distributions, shifted when we applied a likelihood (collected data) to the concept; if we see large shifts, specifically, it can offer insights into the fact that our previous intuition was off.

Within the model’s summary output, we can assess the mean, standard deviation, and confidence intervals for each of the informative priors we defined (our weights). Our first prior weight, the intercept, had a moderately positive mean with a somewhat equal spread and wide confidence interval; this differed slightly from our prior belief that the average EPA would hover around 0, informing us that the impact of the collected data suggests that the average EPA prediction was positively oriented. In terms of the specific yard line feature, we found that the mean was near 0 with an even smaller spread, and a narrow confidence interval; this differed from our prior belief of a mean around 1, as we believed that a higher yard line would relate to a higher EPA prediction, but this difference tells us that collected data in the model informs us that this feature has relatively no effect on EPA. We then believed that offensive yards remaining would have a mean of -0.5, where greater yards remaining would lead to a smaller EPA; we found that the model relatively validated this fact with a mean of -0.43, a smaller spread, and a tight confidence interval, which suggests that collected data did not lead to a large departure from our prior belief (but had an influence of a slightly lesser extent). For the given down, we believed that it would overall decrease the EPA prediction, which the model maintained very closely but with a wider confidence interval. The categorical play type differed more from our prior belief that it would not influence EPA, with the model finding that it would have a positive effect (~0.5 coefficient) with a lower spread but wide confidence interval.

With larger departures from the prior occurring within the intercept, yard line, and play type features, this conveys the fact that the model learned from the data and was not overreliant on the predefined prior information. In addition, we also see narrower spreads in these instances which suggests that the data was somewhat informative for these features. However, with the offensive yards remaining and down features, we saw that the importance/coefficients of these models did not differ greatly from our prior beliefs. This suggests to us that the model did not necessarily learn from collected data (the data did not have much of a differing influence): for example, the feature might not be strongly related with EPA or the collected data might be insufficient.

Predictive Importance & Model Performance:



To measure the predictive importance and the model's overall importance, we implemented cross validation, where we trained our model on 80% of the data for the 2019 season, between our predictive and target (EPA) features.

We also considered a check of the general posterior predictive distribution for EPA. Based on this distribution, we can see that the model's prediction is around -0.008 (basically equivalent to 0) with a spread equal to roughly 1. This conveys that the posterior conventionally follows a similar shape and spread with how EPA was originally distributed, even though slight differences exist within both values but more-so in the spread which signifies our model's limitations; these spread differences can be seen in the density plots for the actual EPA data and our predicted EPA data (a PDF calculated using the samples from the posterior). The data looks to be somewhat representative, but more statistical factors relating to cross validation must be considered.

After inversely scaling our model's predictions to the NFL dataset scale, they were still outside the bounds of the test set's (ranging from -0.5 to 0.24 versus the test set's -9.6 to 6.5 range). This could be due to outliers, or could hint at the fact that our model did not learn the data well enough.

We then found the R-Squared value, equal to 0.002, which explains to us that our model's variations in the predicted EPA were only able to explain 0.2% of the variation of the actual EPA. Then, to assess the problematic nature of large errors present, we interpreted the Mean Squared Error as 138.5% which conveys that the model is an unlikely fit. We also assessed the issue of outliers present, and therefore a skewed MSE by analyzing the Mean Absolute Error, which at 96.5% means that the predictions are off by an average of this scale and our model has not performed well. These statistics prove to us that, although our model found interesting coefficients and could provide further insights into EPA metrics, based on specific play-by-play data, it cannot be truly trusted. The only caveat of this analysis is that, when calculating the mean baseline R-Squared value (we set this equal to the mean of the EPA training set), we found that

our model performed better than that statistic, in which our R-Squared value of 0.002 was greater than -.0004.

Conclusion & Future Work:

All in all, our highest performing MLP achieved an accuracy of 30.2% in predicting one of six NFL play types from pre-snap situational data, noticeably outperforming both random choice (16.7%) and a naive baseline of always predicting the most common class (25%). While this accuracy appears low, it reflects the complexity of play-calling and the amount of human involvement required. The model's primary confusion between directionally equivalent plays considering our data (short left vs. short right passes) and strategically similar plays (short passes vs. runs) demonstrates that the model has at least some underlying understanding of offensive intent from situational data. It allows us to conclude that even sparse pre-snap data contains some value in being able to signal play calls or play calling patterns.

In the lens of future work, improvements to the MLP would require the incorporation of more relevant data that reflects offensive and defensive formations, personnel matchups, coaching focuses, etc. To put it simply, there currently is not enough situational pre-snap data available to use. Furthermore, a more complex model (paired of course with higher computational power that we did not have access to) likely would be able to get a better grasp of the nuanced relationships between play-by-play statistics and play outcomes.

Within the Bayesian Linear Regression approach, it first seems that the model can be relied upon. This can be seen firstly in how the sampling process was successful, as convergence was reached, specifically when analyzing the R-Hat and ESS values. This would allow for the coefficients in the summary output to be trusted. We can therefore concretely support that, based on this output, the model learned in certain instances while failing to do so in others. In the Feature Importance section, it was mentioned that this failure occurred with the offensive yards remaining and down features. It is also problematic that the yard line feature had a mean coefficient near 0, because in theory field position is a major driver of EPA; this hints that the model failed to learn because of linearity constraints or issues with scaling.

From the cross validation results comparison, we can also gauge that the model has a near-zero predictive power. This could be seen directly in the R-Squared value, where only 0.002% of the variation in EPA is explained, and therefore it is only slightly better than guessing the average EPA for each play. In terms of the disparity between prediction and test set EPA ranges, the predicted range being much smaller suggests that the model is underfitting and not properly reacting to the chosen features. This could be driven by nonlinear relationships in the features or the complexity of play-by-play data, such as how it is difficult in nature to predict EPA from a 3rd down, for example. Despite convergence proving that sampling was valid, the poor model fit proves that the predictions cannot be relied upon for analytical insight. It is likely

that the model, especially for the features previously mentioned, is overreliant on our defined prior beliefs.

Overall, a Bayesian approach can still be useful in predicting EPA, but it would require significant modifications, as determined by our model's results. The first potential issue is linearity, especially within relationships between field position and EPA, for example; a way to navigate this in the future is to experiment by adding a polynomial term to exhibit this more effectively. To improve model quality and performance, we could also experiment by adding/removing features (we tried the removal, but it barely modified the R-Squared value). We could also consider only the numerical continuous variables: the specific yard line, the offensive team's amount of yards before the end zone, and the amount of seconds remaining in the quarter. The categorical features have multiple weights per the true/false conditions, so maybe this is confusing the model by introducing mathematically difficult dependencies (with identifiability), and it would learn the continuous features better if these were omitted. The final issue is with scaling (Min-Max Scaling on x-features and Standardization on EPA), as this might have contributed to low predictive power by making it harder for the model to distinguish away from the prior coefficients. To experiment with solving this issue, an approach might be to standardize the x-features instead of Min-Max Scaling, as standardization makes it easier for the model to reason about certain priors (e.g. when $\sigma=1$).

Appendix:

- MLP:

- Architecture & Setup -

```
set.seed(40) # maintain state

eta <- 0.1 # step size
q <- 24 # hidden nodes

n <- nrow(X_train) # number of samples
d <- ncol(X_train) # number of features + 1 (bias term)
k <- ncol(Y_train) # number of classes

W1 <- matrix(rnorm(d * q), nrow = d, ncol = q) # initial W1
W2 <- matrix(rnorm(q * k), nrow = q, ncol = k) # initial W2

# leaky relu
leaky_relu <- function(x, alpha = 0.01) {
  ifelse(x > 0, x, alpha * x)
}

# softmax
softmax <- function(x) {
  x_shift <- x - apply(x, 1, max) # infinity values were causing errors
  exp_x <- exp(x_shift)
  exp_x / rowSums(exp_x)
}

# forward pass func
forward_pass <- function(X, W1, W2) {
  Hidden_input <- X %>% W1
  Hidden_output <- leaky_relu(Hidden_input)
  Output_input <- Hidden_output %>% W2
  Y_pred <- softmax(Output_input)

  list(Y_pred = Y_pred, Hidden_output = Hidden_output,
       Hidden_input = Hidden_input)
}
...

```

- Backpropagation -

```
errors <- numeric() # to track errors over epochs
epochs <- 500 # number of epochs

# backprop
for (epoch in 1:epochs) {
  # forward pass
  fwd <- forward_pass(X_train, W1, W2)

  # gradients
  dw2 <- t(fwd$Hidden_output) %>% (fwd$Y_pred - Y_train) / n
  dw1 <- t(X_train) %>% (((fwd$Y_pred - Y_train) %>% t(w2)) *
                        ifelse(fwd$Hidden_input > 0, 1, 0.01) ) / n

  # weight updates
  W2 <- W2 - eta * dw2
  W1 <- W1 - eta * dw1

  # calculate and track loss
  epsilon <- 1e-15 # to avoid log(0) error
  L <- -mean(rowSums(Y_train * log(fwd$Y_pred + epsilon)))
  errors <- c(errors, L)
}

```

- **Bayesian Linear Regression:**
 - **Model Implementation & Sampling -**

```
[247]: # Create the model
df_down = x_train.iloc[:, [3, 4, 5, 6]]
df_playtype = x_train.iloc[:, [7, 8, 9, 10, 11, 12, 13]]
df_playtype = x_train.iloc[:, [7, 8, 9]]

with pymc.Model() as epa_norm:
    # Weight Priors
    w0 = pymc.Normal("w0", mu=0, sigma=0.5)
    w_yardline = pymc.Normal("w_yardline", mu=1, sigma=0.5)
    w_ydstogo = pymc.Normal("w_ydstogo", mu=-0.5, sigma=0.3)
    w_qtrsecs = pymc.Normal("w_qtrsecs", mu=0.5, sigma=0.5)
    w_down = pymc.Normal("w_down", mu=-1, sigma=0.5, shape=df_down.shape[1])
    w_playtype = pymc.Normal("w_playtype", mu=0, sigma=1, shape=df_playtype.shape[1])

    # Calculate Mu (weight priors * actual data)
    mu = (
        w0 +
        w_yardline * x_train["yardline_100"] +
        w_ydstogo * x_train["ydstogo"] +
        w_qtrsecs * x_train["quarter_seconds_remaining"] +
        w_down @ df_down.T +
        w_playtype @ df_playtype.T
    )

    # Define a prior for unknown covariance matrix
    sigma = pymc.InverseGamma("sigma", alpha=1.5, beta=1)

    epa_data = pymc.Normal("epa_data", mu=mu, sigma=sigma,
                           observed=y_train)

    # Obtain samples by estimating conditional posterior
    post_samples = pymc.jax.sample_numpyro_nuts(
        draws=1000,
        tune=180, # burn
        chains=4, # need these 4 to converge
        target_accept=0.95,
        #return_inferencedata=True
    )
```

- **Posterior Predictive Distribution -**

```
[200]: # Visualize the Posterior Predictive samples
with epa_norm:
    preds_norm = pymc.sample_posterior_predictive(post_samples, var_names=["epa_data"])

# Plot the histogram
plt.hist(preds_norm.posterior_predictive["epa_data"][:, :, 0].values.ravel(), bins=30)
plt.title("EPA Posterior Predictive Distribution (Gaussian Dist.)")
plt.xlabel("EPA")
plt.show()
```

- **Calculating Predictions -**

```
[252]: # Find the dimensions to reshape categorical variables by
n_samples = post_samples.posterior["w0"].values.size # chains * draws

# Obtain test splits for categorical data
df_down_test = x_test.iloc[:, [3, 4, 5, 6]]
df_playtype_test = x_test.iloc[:, [7, 8, 9]]

# Reshape all parameters for predictions
w0_flat = post_samples.posterior["w0"].values.flatten()
w_yardline_flat = post_samples.posterior["w_yardline"].values.flatten()
w_ydstogo_flat = post_samples.posterior["w_ydstogo"].values.flatten()
w_qtrsecs_flat = post_samples.posterior["w_qtrsecs"].values.flatten()
w_down_flat = post_samples.posterior["w_down"].values.reshape(n_samples, -1)
w_playtype_flat = post_samples.posterior["w_playtype"].values.reshape(n_samples, -1)

# Calculate predictions for posterior samples
mu_test = (
    w0_flat[:, None] +
    w_yardline_flat[:, None] * x_test["yardline_100"].values +
    w_ydstogo_flat[:, None] * x_test["ydstogo"].values +
    w_qtrsecs_flat[:, None] * x_test["quarter_seconds_remaining"].values +
    w_down_flat @ df_down_test.values.T +
    w_playtype_flat @ df_playtype_test.values.T
)

# Obtain the predictions to compare with the actual (it is a distribution, so average it)
# Average across all posterior samples
y_pred_standardized = mu_test.mean(axis=0)
y_pred = std_scaler.inverse_transform(y_pred_standardized.reshape(-1, 1)).flatten()

# After making predictions, inverse-transform y_test
y_test = std_scaler.inverse_transform(y_test.reshape(-1, 1)).flatten()
```

Bibliography:

- Cochran, Jason J. 2010. "The Emergence of Sports Analytics." *The Emergence of Sports Analytics* (INFORMS).
<https://pubsonline.informs.org/doi/10.1287/LYTX.2010.01.06/full/>
- Fernandes, C., R. Yakubov, Y. Li, A. Prasad, and T. Chan. 2019. "Predicting Plays in the National Football League." *Journal of Sports Analytics* 6: 1–9.
<https://journals.sagepub.com/doi/10.3233/JSA-190348>
- Husowitz, Barry, Mark Mixer, and Steven Morrow. 2024. "NFL Data Analytics and Predictions Using Machine Learning." Living reference work entry in *Handbook of Visual, Experimental and Computational Mathematics*, pp. 1–21. October 1, 2024. Springer.
https://link.springer.com/rwe/10.1007/978-3-030-93954-0_22-1.
- Lock, Daniel, and Dan Nettleton. 2014. "Using Random Forests to Estimate Win Probability Before Each Play of an NFL Game." *Journal of Quantitative Analysis in Sports* 10, no. 2 (June 2014): 197-205.
https://www.researchgate.net/publication/274325256_Using_random_forests_to_estimate_win_probability_before_each_play_of_an_NFL_game
- London Post. 2024. "The Analytics Revolution in the NFL." *London Post*, March 2024.
<https://london-post.co.uk/the-analytics-revolution-in-the-nfl/>
- Pilgrim, Ross. 2023. "Evolution and Usage of Statistics in MLB." *Baseball Egg*, July 3, 2023.
https://baseballegg.com/2023/07/03/evolution-and-usage-of-statistics-in-mlb/#google_vignette
- Yamani, Esha, Dhvani Patel, Vani Singh, and Kavin Indirajith. 2025. "Predicting NFL Plays with Deep Learning: Turning Game Data into Smart Insights." *Medium*, May 12, 2025.
<https://medium.com/@esha.yamani/predicting-nfl-plays-with-deep-learning-turning-game-data-into-smart-insights-a76f502903a1>
- Yurko, Ronald, Ventura, Samuel, & Horowitz, Maskim. 2019. nflWAR: a reproducible method for offensive player evaluation in football. *Journal of Quantitative Analysis in Sports*. 15(3), 163-183. April 16, 2019.
<https://doi.org/10.1515/jqas-2018-0010>.